Crash-safety discipline in ConnectRedirectClassify

A bug in most code crashes a process. A bug in this function — kernel-mode, on the network data path, invoked on every outbound connect() that matches a redirect filter — bugchecks the whole machine. This document is an honest accounting of what has actually been done to keep that from happening, and equally honest about what has **not** been done, so a reviewer (or a future version of this document) isn't misled into treating "compiled and reasoned about" as "proven safe."

Status as of this writing: **compiled against WDK 10.0.26100, never loaded, never exercised.** Everything below is design-time and review-time discipline, not verified runtime behavior.

What's actually been done

The safe action is set first, unconditionally, before any logic runs. classifyOut->actionType = FWP_ACTION_PERMIT is the very first statement in the function. Every one of its many early-return paths — shutting down, no layerData, no redirect handle, rights already claimed, no target configured, loop-guard hit, handle-acquisition failure, writable-pointer failure — is simply return; with nothing else to undo. A future bug that adds a wrong early return, or removes a check, still leaves the connection permitted unchanged. There is no path, reachable by any combination of these checks, that leaves classifyOut in an unset or ambiguous state.

The function cannot block, structurally, not just by convention. Its entire action vocabulary is binary: modify the connect request and permit, or permit unchanged. FWP_ACTION_BLOCK (or any denying action) never appears in this function at all. The worst a logic bug here can do is misroute a connection or fail to redirect it — never sever it. This is a narrower failure mode than the existing SNI/STREAM callout, which has to be able to block by design and carries that risk deliberately.

No heap allocation anywhere in this path. ⚠ Slated to change — see issue 9. Keeping the redirect-context mechanism (decided 2026-10-15) means the driver must ExAllocatePoolWithTag a context per redirected flow, so the shipped design will not satisfy this bullet. It holds for the code as it stands today and is recorded here because the reasoning still applies to everything else in the function. The SNI/STREAM classify function calls ExAllocatePool2 for pended-flow records and held stream segments, and has to handle OOM on each one. ConnectRedirectClassify allocates nothing — no pool tag, no allocation failure to handle, no use-after-free surface from a buffer freed on one path and read on another. Fewer resources touched means fewer ways to get resource lifetime wrong.

Fail-open is checked explicitly, early, and before anything is touched: g_ShuttingDown, layerData == NULL, g_RedirectHandle == NULL, FWPS_RIGHT_ACTION_WRITE not held, {pid, port} unpacking to zero, the redirect-loop-guard state, FwpsAcquireClassifyHandle0 failure, and FwpsAcquireWritableLayerDataPointer0 failure — every one of these is a guard clause that returns before any mutation. None of them are "handle the error and continue"; they are all "stop here."

The struct layout and API semantics were checked against the real installed WDK headers, not transcribed from memory or the design doc's prose — and two things the prose got wrong were caught this way: localRedirectTargetPID is a DWORD, not a HANDLE; and there are two distinct "redirected by self" enum members (FWPS_CONNECTION_REDIRECTED_BY_SELF and FWPS_CONNECTION_PREVIOUSLY_REDIRECTED_BY_SELF), both now handled by the loop guard. A wrong field type or a missed enum member in this category of API is a type-confusion or silent-logic bug, not something that fails loudly at compile time. in6addr_loopback was deliberately avoided (it pulls in a separate link library — the same reason Windows' own IN6_IS_ADDR_LOOPBACK avoids it) in favor of constructing ::1 by hand, removing a plausible link-time failure mode entirely.

No new global mutable state beyond one read-mostly handle. The only process-wide state this feature owns is g_RedirectHandle — created once, at PASSIVE_LEVEL, during RegisterCallouts, and destroyed once at unload. There is no counter, table, or lock introduced by this feature, and — since the per-agent target moved from two global variables to each matched filter's own context field — there is now **no write path into this feature's runtime state from inside a classify invocation at all**. Two CPUs running this classify function concurrently have nothing of this feature's own to race on.

Build-level discipline matches the existing driver's bar. /kernel /GS /W4 /WX — kernel-mode compilation, stack-protection cookies, and every warning promoted to an error, so an uninitialized read or an implicit narrowing conversion fails the build instead of shipping quietly. This isn't new for this feature; it's the same build.cmd flags the whole file already builds under.

Separable from the already-further-along SNI/STREAM datapath. The two features share only the WFP provider/sublayer GUIDs (an organizational grouping) and the device/module they're compiled into. No runtime state, lock, or data structure is shared between them.

What has explicitly NOT been done

- **Never loaded.** Not on a test-signed VM, not under a debugger, not once. Everything above is static review and type-checking, not observed behavior.
- **No Driver Verifier pass.** The existing SNI driver's own validation checklist still lists this as open for its half of the file; the redirect half hasn't reached even that stage.
- **No fuzzing, no concurrency/load test.** Nothing has exercised two simultaneous redirected connections, a listener restart mid-flow, or a malformed/adversarial connect attempt against this code.
- **IRQL correctness is "follows the documented API contracts," not independently verified.** Every call used (FwpsAcquireClassifyHandle0, FwpsAcquireWritableLayerDataPointer0, FwpsQueryConnectionRedirectState0, etc.) is used within its documented _IRQL_requires_max_ bound, but no tool has confirmed that at runtime.

- **Code signing is unrelated to all of the above and doesn't substitute for it.** EV/attestation/WHQL signing proves provenance, not correctness. A perfectly signed driver with a logic bug still bugchecks the machine.
- **Test-signing and loading are scoped to a disposable VM, deliberately never a primary machine** — precisely because none of the above has been verified yet, and the failure mode if something's wrong is "the machine goes down," not "the process restarts."

The one thing this document can't claim

Nothing above defends against memory corruption that originates **outside** this function's own code — a wild write from unrelated kernel code elsewhere in this module, or another driver entirely, could still corrupt g_RedirectHandle or any other global. That's a general property of running in kernel mode at all, not something a single classify function can guarantee for itself. The measures above are about this function's own logic being unable to produce a bad outcome on its own, not about the machine being safe from every other actor in Ring 0.

Before this is loaded anywhere, even the disposable VM

1. Attach WinDbg (kernel) before the first sc start, same as the README already requires for the SNI half.
2. Run under Driver Verifier (WFP + pool tracking) for the whole pass, not just the SNI callouts.
3. Validate the mechanism against a single throwaway 127.0.0.1 echo listener — one connection, manually driven — before anything resembling real agent traffic. **Follow README.md's "Exercising the CONNECT_REDIRECT half end to end"** rather than improvising the setup: a filter only installs once the rule, a *genuinely listening* {pid, port}, and an IPv4-resolved destination all line up, and each of those failing looks identical from the outside ("nothing happened"). Most of the ways to waste an afternoon on this are in that list, not in the kernel.
4. Confirm the unload path (filter delete, callout unregister, redirect handle destroy) leaves no dangling WFP objects by re-running the whole sequence (load → exercise → unload → load again) rather than loading once and calling it done.

Open issues found in review (2026-10-14)

Scope note: this document is otherwise about ConnectRedirectClassify itself. These are review findings on the **whole feature** — driver, daemon, schema and rollout — recorded here because the commit message points at this file as the home for "the open items before this leaves prototype status." None of them is a kernel crash-safety defect; the classify function's own fail-open discipline above still holds. Ordered by severity.

Status: **#2, #3, #4, #6 and #8 are fixed**, **#5 is partially fixed** (the silent-gap half; real IPv6 coverage is left open as a feature), see the notes inline. **#1 is deliberately held** — the part that affects already-deployed wardens cannot be fixed in code at all, and the structural fix (relocating redirect policy to its own top-level settings key, as ProcessTraceConfig did) is a schema decision for the team rather than a unilateral refactor; #4's doc change records the

rollout constraint in the meantime. **#7 remains open** and needs the VM. **#9 is open and gates the end-to-end path** — it is the reason "stage 2" is a kernel change before it is a listener change.

Separately from this list, the ListenerMapping producer the sequence diagram called "the single hard blocker" is **now implemented**: ai-gateway reports a serviceable agent's bound {agent_id, pid, port} on AgentStatus, and subscribers::ai_broker_launch republishes it for this feature. The ZSAI_NETFLT_TEST_LISTENERS stub still overrides it, so a VM test needs no running gateway.

⚠ **#8 was found while fixing #6 and is the most severe of the set** — a redirect rule also installed a *block* filter on the traffic it was meant to redirect. Read that entry before trusting any earlier description of this feature as "cannot sever a connection."

1. The new redirect action and redirect_agent field need endpoint-first rollout, and nothing says so

EgressRule is #[serde(deny_unknown_fields)] and RuleAction is a strict lowercase enum with no #[serde(other)] fallback. This commit adds a fourth variant (redirect) **and** a new field (redirect_agent) to that struct. docs/settings-keys.md already documents this exact hazard for the *previous* variant added:

> ⚠ action is a strict enum: a warden predating "audit" fails to parse the rule and apply then keeps its PREVIOUS config, so "audit" needs **endpoint-first rollout** before any console emits it.

So the moment a console emits a redirect rule, every warden predating this commit fails to parse the rule and **keeps its previous network_egress config** — it silently stops picking up policy changes (new block rules included) for as long as the tenant's policy contains a redirect rule. On a mixed-version fleet that is a quiet, per-device policy freeze, not a loud failure. Compare ProcessTraceConfig, which this codebase deliberately made a **top-level** settings key rather than a network_egress field for precisely this reason (config.rs's own comment: "a released warden would reject the WHOLE network policy, block rules included, over one key it does not know").

Before this ships: state the rollout ordering explicitly, and decide whether redirect_agent belongs under network_egress at all given that precedent.

2. A stale ListenerMapping fails CLOSED, not open — FIXED 2026-10-15

compute_redirect_filters resolves a listener with a bare listeners.iter().find(|l| l.agent_id == agent_id). Nothing anywhere validates that the pid is still alive or the port still bound. The installed filter keeps rewriting the agent's connects to 127.0.0.1:<port>, so when the listener dies or restarts on a new port the agent's CAPI connections are redirected to a closed port and get RST/ECONNREFUSED — they do not fall through to the real destination.

This is the one place the feature's stated guarantee does not hold end to end. "The worst a logic bug here can do is misroute a connection or fail to redirect it — never sever it" is true of the **WFP action** (it is always FWP_ACTION_PERMIT) and false of the **observable outcome**: a permit to a dead loopback port severs the connection just as effectively as a block.

Under the ZSAI_NETFLT_TEST_LISTENERS stub this is unbounded — the stub explicitly never live-reloads, so a relaunched test listener leaves a permanently wrong target until the daemon restarts. With a real producer it narrows to the window between listener death and the next reconcile tick, but it does not close.

Fix applied: net_egress::live_listeners now filters the mapping set before compute_redirect_filters sees it, keeping only mappings whose {pid, port} still owns a listening IPv4 TCP socket (GetExtendedTcpTable/TCP_TABLE_OWNER_PID_LISTENER). A mapping that fails the check is dropped with a warning, so no filter is installed and the agent's traffic reaches its real destination. If the socket table itself can't be read, *every* mapping is dropped rather than trusted — unable-to-confirm is treated as not-live. The keep/drop half (retain_live) is pure and unit -tested on every platform; only the enumeration is Windows-gated.

What the fix does not do, stated plainly: it shrinks the window to one reconcile tick, it does not close it. Live at reconcile time is not live at connect time, and no in-kernel redirect to a userspace listener can close that gap — the driver cannot probe reachability at classify time (DISPATCH_LEVEL, no blocking). It also checks (pid, port) only, not the bound address, so a listener bound exclusively to a non-loopback address would pass and still refuse the redirected connection; ai-gateway binds loopback by design, so this avoids depending on a second byte-order assumption. A stable listener port would still be the stronger answer and is left as a design question.

⚠ The FFI compiles (verified against windows-sys 0.52 by cross-checking for x86_64-pc-windows-msvc) but has **never run** — the two-pass buffer sizing and the dwLocalPort byte-order handling are unexercised. Confirm listening_tcp_sockets() actually returns the expected pairs on the VM before trusting the guard, because a wrong read here fails in the safe direction (no redirect at all) and would look like "the feature doesn't work" rather than like a bug.

3. Three new counters landed inside the dedup-hashed inventory payload — FIXED 2026-10-15

EnforcementStatus is embedded as "enforcement" in the network_acl inventory event (src/tasks/network_acl/event.rs), which goes out through Storage::write_observation — i.e. the **enriched payload is hashed** per (task_name, scope_key). suppressed_counts is explicitly excluded from that hash; these three new fields are not. Per AGENTS.md's inventory-dedup section, "adding a hashed key re-emits every row once per device", so this upgrade re-emits the whole network_acl row across the fleet, on the strict-FIFO Ordered lane, to carry three counters the commit itself describes as permanently zero today.

Separately, these are enforcement *runtime* counters on an *inventory* row — the same shape AGENTS.md warns about ("HashChanged with a constant items count tick after tick means a non-inventory field is in the hash"). The three pre-existing block/allow counters have the same property, so this is inherited rather than new, but it doubles down on it.

Fix applied: the three key names are now in storage::canonical's VOLATILE_HASH_KEYS, beside last_reconcile_at — which is itself an EnforcementStatus field already excluded for the same reason. Because the exclusion ships in the same build that adds the fields, the hash never changes and the one-time fleet-wide re-emit does not happen at all. degraded is deliberately

left *in* the hash, so a redirect pass that starts failing still re-emits the row. The three pre-existing block/allow counters are left alone: tightening those changes established emit behaviour for a shipped field, which is a separate decision rather than part of this fix.

4. The shipped schema docs do not mention redirect or redirect_agent — FIXED 2026-10-15

docs/settings-keys.md — which AGENTS.md designates as the per-key reference for the tenant-configurable keyspace — still said "Rule actions are block | allow | audit", and docs/network-egress.md's rule-schema block carried neither the new action nor the new field.

Fix applied: both are documented now. docs/network-egress.md gains the field in the schema block plus a "Redirect action" section covering the different WFP layer, the required redirect_agent, the stricter destination-AND-process scope, its absence from the allow/block/audit precedence, the IPv4-only and Windows-only limits, and a plain prototype-not-loadable status marker. docs/settings-keys.md's strict-enum ⚠ now names redirect/redirect_agent alongside "audit" and states that **no console may emit either until the fleet carries a build that knows them** — which is also the only mitigation available for issue 1, recorded where whoever wires the console will actually read it.

5. Redirect filters can never cover an IPv6-resolved destination — ⚠ PARTIALLY FIXED 2026-10-15 (made visible, not closed)

TargetKind and DnsMapping are Ipv4Addr-only, so a destination observed only via an AAAA answer produces no filter — and compute_redirect_filters cannot tell that case apart from "not resolved yet", so it emits no warning either. The driver registers a complete ALE_CONNECT_REDIRECT_V6 callout and smoke-test filter that the daemon has no way to drive. The IPv4-only scope is inherited from the rest of net_egress (and documented there), but for an *enforcement* path the silent-gap reading matters more: on a dual-stack network preferring AAAA, the mechanism never engages and nothing says so.

What was fixed: the silence, not the gap. Actually covering IPv6 means giving TargetKind/DnsMapping v6 variants, recording AAAA answers in the DNS sensor, and adding an ALE_CONNECT_REDIRECT_V6 filter path — i.e. implementing IPv6 support across all of net_egress, including the well-tested block/allow path that shares those types. That is a feature with its own blast radius, not a fix to this prototype, and it is left open.

What is closed is the "nothing says so" half. compute_redirect_filters_with_diagnostics now returns, alongside the filter set, one line per redirect rule that is enforcing yet resolved to nothing, naming the empty dimension; run_session logs it change-gated, the same way skipped_rules is handled. For the destination case the reason states the limitation outright ("resolves A records only — a destination reached over IPv6 never resolves here") rather than claiming to have diagnosed it, because the daemon genuinely cannot distinguish AAAA-only from not-resolved-yet: no AAAA answer is ever recorded, so both arrive here as an empty target set.

Deliberately **not** added to EnforcementStatus: that struct rides in the dedup-hashed network_acl inventory payload, and a field flipping on listener churn would re-emit the inventory row —

the problem issue 3 just fixed. Log-only keeps the diagnostic out of the hash. The already-present redirect_desired_filters: 0 remains the machine-readable signal.

A side effect worth noting: this also surfaces issue 6's case (a rule with no redirect_agent), which skipped_unconstrained_rules misses entirely. That is a diagnostic, not the validation fix issue 6 asks for — the rule still passes constrains_scope().

6. constrains_scope() does not require redirect_agent — FIXED 2026-10-15

The RuleAction::Redirect arm checked destination AND process but not redirect_agent, while compute_redirect_filters treats a missing redirect_agent as fatal to the rule. skipped_unconstrained_rules — the module's only "will this rule ever do anything" warning — is defined purely as !r.constrains_scope(), so a redirect rule missing its agent id passed validation, warned about nothing, and installed nothing.

Fix applied: the Redirect arm now requires all three. The skipped_unconstrained_rules warning text says so explicitly, since that message previously only talked about destination/process/user and would have read as wrong for this case. The inert-redirect diagnostic added for issue 5 no longer reports the missing-agent case — constrains_scope catches it first, and warning about one rule in two different messages is worse than warning once.

8. A redirect rule also installed a BLOCK filter — FIXED 2026-10-15

Found while checking issue 6's callers, and **the most severe defect in this feature** — numbered last only to keep the references in earlier commits stable.

compute_filters_with_conflicts_and_labels skipped only RuleAction::Audit before deriving let permit = matches!(rule.action, RuleAction::Allow). A Redirect rule therefore fell straight through with permit == false and emitted a **block** filter at ALE_AUTH_CONNECT for exactly the destination + process it was supposed to redirect. Confirmed by probe, not inferred:

```
compute_filters -> 1 filter(s): FilterSpec { permit: false,
   target: Some(Host(1.2.3.4)), port: Some(443),
   app_path: Some("C:\\code.exe"), user_sid: None }
```

The failure is worst exactly where the redirect cannot work. On a device without the zsai-netflt driver — which is every device today, since it is unpackaged and unsigned — the redirect filter add fails ("no such callout") while this block installs perfectly, so authoring a redirect rule would simply **block the agent's traffic**, the precise inverse of the intent. It also defeated issue 2's fix: dropping the redirect filter for a dead listener so traffic goes direct is pointless if a block filter is still severing it.

Fix applied: Redirect is now skipped in the same place Audit is, with two tests pinning it — one that a redirect rule installs nothing at that layer, one that it does not disturb neighbouring block/allow rules. The evaluator side was already correct (RuleAction::Redirect => {} contributes nothing to the allow/block/audit precedence).

The general lesson for this file: RuleAction is matched non-exhaustively in more than one place, and a new variant is assumed to be block-shaped by default. Any further variant needs every matches!(rule.action, ...) site audited, not just the ones the compiler points at.

9. The listener cannot learn the original destination — the driver supplies no redirect context

Found 2026-10-15 while preparing the end-to-end ("stage 2") work, and it moves that work from "a listener change" to "a kernel change first".

ConnectRedirectClassify sets exactly two fields on the connect request: localRedirectTargetPID and localRedirectHandle. It does **not** set localRedirectContext / localRedirectContextSize.

That field is the documented channel by which a redirected connection's *original* destination reaches the user-mode proxy: the callout stores it, and the proxy retrieves it after accept() via SIO_QUERY_WFP_CONNECTION_REDIRECT_CONTEXT (Win8+; the field is explicitly unavailable on Windows 7). SIO_QUERY_WFP_CONNECTION_REDIRECT_RECORDS — which the sequence diagram's step 10 names — returns the opaque records used to *relate* the two connections for the loop guard; it is not how the address comes back.

So as the driver stands, implementing the user-mode query would retrieve nothing. Reported symptom for exactly this shape is WSAEINVAL from the context query even though redirection itself works. The ai-gateway listener has no other way to recover the destination either: its existing paths read it from a CONNECT line or from SNI/Host, neither of which a transparently redirected flow carries.

DECIDED 2026-10-15: keep the context mechanism for the shipped design. The SNI alternative is a validation shortcut only — see the correction below for why it is not equivalent. What follows is therefore a specification, not an open question.

Allocation semantics — resolved, no longer speculative. Microsoft's FWPS_CONNECT_REQUEST0 reference settles the lifetime question that was open here:

> the callout driver allocates localRedirectContext with ExAllocatePoolWithTag … Starting with Windows 8, memory allocated for localRedirectContext will have its ownership taken by WFP, and will be freed when the proxied flow is removed.

So the driver **allocates per redirected flow and must never free it** — ownership transfers at handoff. The earlier "does WFP copy it, or must the buffer outlive the call?" framing dissolves: WFP keeps the pointer. localRedirectContextSize must match the allocation exactly.

Three consequences to carry into the implementation:

1. **The "no heap allocation anywhere in this path" claim above must narrow.** It is one of the stated reasons this classify function is argued to be lower-risk than the SNI/STREAM one, and the shipped design will no longer satisfy it. Amend that bullet when the code lands rather than leaving the document asserting a property the feature does not have.
2. **An allocation failure must fail open.** The existing discipline is that every early return leaves FWP_ACTION_PERMIT with nothing to undo; a failed ExAllocatePoolWithTag has to join that set — return without redirecting, never block, never half-apply. This is the first failure mode in this function that is not simply a missing precondition.

3. **The pool tag and charge** should match the driver's existing convention for its SNI allocations so a leak is attributable to this feature rather than to the driver generally.

Implementation spec (decided 2026-10-15)

Settled in discussion, recorded so none of it is re-derived or re-argued:

Question	Decision
Carry the destination in localRedirectContext?	**Yes** — keep it. SNI is a validation shortcut only.
Index into a config-file table instead of the destination?	**Skipped for now.** Viable, but an index is only meaningful against a table *version*, and a watcher on each side gives two sources of truth that can skew.
Port-per-destination (one loopback port per configured host)	**Considered, not taken.** It needs *no* driver change and no allocation at all, because the existing filter->context already carries a per-filter port. Kept on the record as the fallback if the allocation path proves troublesome — it trades N listener ports for zero kernel allocation.
Sub-allocate from one large driver-owned pool?	**Ruled out — incompatible.** WFP frees *that exact pointer*; an offset into a private slab means freeing a non-pool-base address, i.e. an immediate bugcheck.
Lookaside list?	**Legal but pointless.** Each entry is a real pool block, so handing one over is safe, but WFP frees it to the general pool rather than back to the list — zero reuse, degenerates to plain per-flow allocation.
Upper size limit on the allocation?	**Skipped.** The content is driver-generated, not client-supplied, so a cap defends only against our own size arithmetic.
Ceiling on outstanding contexts?	**Not implementable.** There is no release signal: WFP frees silently, the redirect callouts register flowDeleteFn == NULL, and ALE_CONNECT_REDIRECT is not a flow-established layer, so nothing notifies us when a context goes away. A counter cannot be decremented.

Total exposure is therefore (payload size) × (concurrent redirected flows), with no enforceable bound on the second factor. In practice it is bounded by filter scope instead: one process, a handful of IPs, port 443 — tens of concurrent flows, so kilobytes. The risk in this path is the correctness of the allocation call and its failure path, not memory volume.

Requirements for whoever writes it:

1. **Non-paged pool.** The field docs specify no pool type (a real gap), but classify can run at DISPATCH_LEVEL, so paged memory would fault at raised IRQL. POOL_FLAG_NON_PAGED.
2. **Own pool tag**, so a leak is attributable to this feature rather than to the driver generally.
3. **Allocation failure joins the fail-open set** — return without redirecting, never block, never half-apply. It becomes the ninth early return and the first that is a *resource* failure rather than a missing precondition.
4. **Never free it**, and **never read it after FwpsApplyModifiedLayerData0** — ownership has transferred at that point, so touching it afterwards is a use-after-free waiting to happen. The upside of that contract is that the driver has no free path at all, so double-free and use-after-free are structurally absent from our own code.
5. **localRedirectContextSize must match the allocation exactly.**

Two confirmations still owed, both cheap:

- The docs name ExAllocatePoolWithTag; this driver already uses ExAllocatePool2 for its SNI allocations. Both yield pool blocks that ExFreePool can release, so WFP should not care — but confirm rather than discover, and prefer the driver's existing convention for consistency.
- The pool type, since the field documentation is silent on it.

⚠ **Build-sensitivity worth knowing before the VM run.** A developer report on Microsoft Q&A describes a redirect driver that passed on Windows 7, 10 and 11 build 22 000.556 but broke on 11 build 22 621.1265, attributed to added code-integrity checks in AlepValidateConnectRedirectLayerData. Unverified anecdote, but directly relevant: the test rig is Windows Server 2025 (build 26 100), newer than both. If the redirect silently stops applying on a modern build while the classify function itself looks correct, that validation routine is where to look.

On the SNI option specifically — corrected 2026-10-15. An earlier revision of this entry said reading the destination from SNI "may make the whole context mechanism unnecessary for this feature". That was too strong, and conflated two different claims. Stated properly:

SNI is a reasonable way to **unblock stage-2 validation** — the listener terminates TLS and already sniffs the ClientHello, CAPI is HTTPS, so the hostname is there for the taking. It is **not** a replacement for the kernel-supplied destination in a shipped design, for four reasons:

- **The port is lost.** A rule with empty ports is a wildcard across every port; SNI carries none. Always 443 for CAPI, so benign today — but real information loss, not zero.
- **The matched IP is lost.** The filter matched a *resolved* IP. Re-resolving the SNI name can land on a different one (anycast, geo, DNS churn between the resolution and the connect).

- **It is client-asserted where the kernel's value is OS ground truth.** The kernel knows where the connection was actually headed; SNI is what the client says. For an enforcement feature that is a weakening. It is bounded — the filter already restricted redirection to a matched process AND matched IPs, so a lie cannot pull in new traffic, only mislabel traffic already inside that set. But mislabelling *within* the set is exactly the realistic case, because this codebase already treats **shared CDN IPs** as a live hazard: several hostnames behind one matched IP is precisely where "which host was this?" has to be decided, and SNI would be the sole input.
- **ECH removes it.** docs/network-egress-sni.md already records this for the SNI enforcement layer ("no passive observer survives it"), so the fallback inherits a limitation the team has written down elsewhere.

In fairness to the option: within this codebase's own framing SNI *is* the ground truth — network-egress-sni.md calls IP-from-DNS correlation "a guess" and introduced SNI as the improvement on it. The distinction worth holding is observe/block versus forward-onward. If a client lies about SNI to evade a *block*, it still only reaches the IP it was always reaching and the IP-layer filters still apply. If the gateway *forwards* on a lied name, it actively helps the client reach the place it named.

So: use SNI to get a first end-to-end validation moving, and treat the context mechanism as still required for the shipped design — or make the decision to rely on SNI explicitly, with the port/IP/ECH/shared-IP consequences above accepted on the record, rather than by default because the kernel half looked harder.

Not started. No user-mode ioctl code was written, deliberately: with no context to read it would only add a call that always fails, and the kernel half is memory-lifetime code that cannot be compiled or tested from the current dev machine (no WDK on macOS).

7. Same-layer filter evaluation order is unverified

The driver's own two inert (`context == 0`) smoke-test filters and any real operator-added redirect filter all sit at the same layer with `FWP_EMPTY` weight, and `ConnectRedirectClassify` never explicitly clears `classifyOut->rights` or sets an absorb-equivalent flag after a successful `FwpsApplyModifiedLayerData0`. The early-return-on-`context == 0` path makes the inert filters harmless in any order, but "harmless in any order" is a property worth confirming on the VM rather than assuming — it is the one class of defect here that would present as unpredictable behaviour rather than a clean failure to redirect.